

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №4

По дисциплине «Программная инженерия»
Мониторинг и профилирование Java-приложения
Вариант 666999

Выполнил студент группы Р3209
Евграфов Артём Андреевич
Проверил:
Ермаков Михаил Константинович

Санкт-Петербург
2026

Содержание

1	Цель работы	2
2	Техническое задание	2
3	Порядок выполнения работы	2
4	Подключение к JVM для мониторинга	3
5	Исходный код разработанных MBean-классов	3
5.1	MonitoringState	3
5.2	PointStatisticsMBean и PointStatistics	4
5.3	AreaMBean и Area	6
5.4	Регистрация MBean	7
5.5	Интеграция с основным приложением	9
6	Расчёт площади области	11
7	Мониторинг приложения в JConsole	12
7.1	Регистрация и атрибуты MBean	12
7.2	Проверка JMX-уведомления	13
7.3	Время работы виртуальной машины	13
8	Мониторинг и профилирование в VisualVM	13
9	Локализация и устранение проблемы	15
9.1	Алгоритм локализации	15
10	Проверка сборки и работоспособности	16
11	Вывод	16

1. Цель работы

Цель лабораторной работы — реализовать мониторинг веб-приложения из лабораторной работы №3 с помощью JMX, проверить разработанные MBean в JConsole, выполнить мониторинг и профилирование приложения в VisualVM, а также найти и устранить проблему производительности.

2. Техническое задание

Для программы из лабораторной работы №3 необходимо реализовать:

- MBean, считающий общее число установленных пользователем точек и число точек, не попавших в область. При достижении количества точек, кратного 10, MBean должен отправлять уведомление;
- MBean, определяющий площадь получившейся фигуры;
- мониторинг разработанных MBean с помощью JConsole;
- определение времени, прошедшего с момента запуска виртуальной машины;
- мониторинг и профилирование приложения с помощью VisualVM;
- определение класса, экземпляры которого занимают наибольший объём памяти, и устранение выявленной проблемы производительности.

Отчёт должен содержать исходный код разработанных MBean и сопутствующих классов, результаты мониторинга JConsole, результаты мониторинга и профилирования VisualVM, описание найденной проблемы, способ её локализации и результат исправления.

3. Порядок выполнения работы

В приложении реализованы два стандартных MBean. `PointStatistics` хранит статистику проверенных точек и отправляет JMX-уведомление после каждой десятой точки. `Area` вычисляет площадь области по текущему значению радиуса.

MBean регистрируются в платформенном MBeanServer при запуске веб-приложения. При остановке приложения они снимаются с регистрации. Для подключения используется локальный процесс Java-сервера, в котором развёрнуто приложение.

4. Подключение к JVM для мониторинга

При локальном запуске приложение и инструменты JDK работают на одном компьютере. В JConsole в этом случае выбирается локальный процесс Java-сервера. В VisualVM для построения графика пользовательских MBean необходимо установить плагин VisualVM-MBeans, затем выбрать пункт **File > Add JMX Connection** и подключиться к процессу.

При удалённом мониторинге WildFly необходимо открыть management-порт сервера и настроить JMX-подключение. Для учебного стенда может использоваться отдельный порт, например:

Listing 1: Пример параметров запуска удалённого JMX

```
1 -Dcom.sun.management.jmxremote
2 -Dcom.sun.management.jmxremote.authenticate=false
3 -Dcom.sun.management.jmxremote.ssl=false
4 -Dcom.sun.management.jmxremote.port=7203
5 -Dcom.sun.management.jmxremote.rmi.port=7203
6 -Djava.rmi.server.hostname=0.0.0.0
```

В реальной эксплуатации отключать аутентификацию и SSL нельзя. После проброса порта в JConsole и VisualVM указывается JMX-адрес сервера и, если включена защита, учётные данные пользователя WildFly. В VisualVM удалённое подключение выполняется через **File > Add JMX Connection**, а не через обычный пункт добавления удалённого хоста.

5. Исходный код разработанных MBean-классов

Состояние мониторинга вынесено в потокобезопасный класс `MonitoringState`. Это позволяет обоим MBean получать согласованные значения счётчиков и радиуса.

5.1. MonitoringState

Listing 2: MonitoringState.java

```
1 package com.example.lab3.mbean;
2
3 final class MonitoringState {
4
5     private long totalPoints;
6     private long missPoints;
7     private double currentRadius = 1.5;
8
9     synchronized Snapshot registerPoint(boolean hit, double radius)
10     {
11         totalPoints++;
12     }
13 }
```

```

11         currentRadius = radius;
12         if (!hit) {
13             missPoints++;
14         }
15
16         return snapshot(totalPoints % 10 == 0);
17     }
18
19     synchronized Snapshot snapshot() {
20         return snapshot(false);
21     }
22
23     synchronized void updateRadius(double radius) {
24         currentRadius = radius;
25     }
26
27     synchronized void reset() {
28         totalPoints = 0;
29         missPoints = 0;
30     }
31
32     private Snapshot snapshot(boolean tenthPoint) {
33         return new Snapshot(totalPoints, missPoints, currentRadius,
34                             tenthPoint);
35     }
36
37     record Snapshot(long totalPoints, long missPoints, double
38                     currentRadius, boolean tenthPoint) {
39
40         double area() {
41             return currentRadius * currentRadius * (0.75 + Math.PI
42                 / 16.0);
43         }
44     }
45 }

```

5.2. PointStatisticsMBean и PointStatistics

Listing 3: PointStatisticsMBean.java

```

1 package com.example.lab3.mbean;
2
3 public interface PointStatisticsMBean {
4
5     long getTotalPoints();
6
7     long getMissPoints();
8
9     long getHitPoints();
10 }

```

```
11     void reset();
12 }
```

Listing 4: PointStatistics.java

```
1 package com.example.lab3.mbean;
2
3 import javax.management.MBeanNotificationInfo;
4 import javax.management.Notification;
5 import javax.management.NotificationBroadcasterSupport;
6 import java.util.concurrent.atomic.AtomicLong;
7
8 public class PointStatistics extends NotificationBroadcasterSupport
   implements PointStatisticsMBean {
9
10     public static final String TENTH_POINT_NOTIFICATION = "com.
       example.lab4.point.totalMultipleOfTen";
11
12     private final MonitoringState state;
13     private final AtomicLong sequence = new AtomicLong();
14
15     PointStatistics(MonitoringState state) {
16         this.state = state;
17     }
18
19     void registerPoint(boolean hit, double radius) {
20         MonitoringState.Snapshot snapshot = state.registerPoint(hit
           , radius);
21         if (snapshot.tenthPoint()) {
22             Notification notification = new Notification(
23                 TENTH_POINT_NOTIFICATION,
24                 this,
25                 sequence.incrementAndGet(),
26                 System.currentTimeMillis(),
27                 "Total number of user points became multiple of
                   10: " + snapshot.totalPoints()
28             );
29             notification.setUserData("total=" + snapshot.
               totalPoints()
30                 + ", hits=" + (snapshot.totalPoints() -
                   snapshot.missPoints())
31                 + ", misses=" + snapshot.missPoints());
32             sendNotification(notification);
33         }
34     }
35
36     @Override
37     public long getTotalPoints() {
38         return state.snapshot().totalPoints();
39     }
40
41     @Override
```

```

42     public long getMissPoints() {
43         return state.snapshot().missPoints();
44     }
45
46     @Override
47     public long getHitPoints() {
48         MonitoringState.Snapshot snapshot = state.snapshot();
49         return snapshot.totalPoints() - snapshot.missPoints();
50     }
51
52     @Override
53     public void reset() {
54         state.reset();
55     }
56
57     @Override
58     public MBeanNotificationInfo[] getNotificationInfo() {
59         String[] types = {TENTH_POINT_NOTIFICATION};
60         String name = Notification.class.getName();
61         String description = "Notification sent when total user
        point count is multiple of 10";
62         return new MBeanNotificationInfo[]{new
        MBeanNotificationInfo(types, name, description)};
63     }
64 }

```

5.3. AreaMBean и Area

Listing 5: AreaMBean.java

```

1 package com.example.lab3.mbean;
2
3 public interface AreaMBean {
4
5     double getRadius();
6
7     double getArea();
8 }

```

Listing 6: Area.java

```

1 package com.example.lab3.mbean;
2
3 public class Area implements AreaMBean {
4
5     private final MonitoringState state;
6
7     Area(MonitoringState state) {
8         this.state = state;
9     }
10

```

```

11     @Override
12     public double getRadius() {
13         return state.snapshot().currentRadius();
14     }
15
16     @Override
17     public double getArea() {
18         return state.snapshot().area();
19     }
20 }

```

5.4. Регистрация MBean

Listing 7: MonitoringRegistry.java

```

1 package com.example.lab3.mbean;
2
3 import javax.management.JMException;
4 import javax.management.MBeanServer;
5 import javax.management.ObjectName;
6 import java.lang.management.ManagementFactory;
7
8 public final class MonitoringRegistry {
9
10     public static final String POINT_STATISTICS_OBJECT_NAME = "com.
        example.lab4:type=PointStatistics";
11     public static final String AREA_OBJECT_NAME = "com.example.lab4
        :type=Area";
12
13     private static final MonitoringState STATE = new
        MonitoringState();
14     private static final PointStatistics POINT_STATISTICS = new
        PointStatistics(STATE);
15     private static final Area AREA = new Area(STATE);
16
17     private MonitoringRegistry() {
18     }
19
20     public static synchronized void registerMBeans() {
21         MBeanServer server = ManagementFactory.
            getPlatformMBeanServer();
22         try {
23             register(server, new ObjectName(
                POINT_STATISTICS_OBJECT_NAME), POINT_STATISTICS);
24             register(server, new ObjectName(AREA_OBJECT_NAME), AREA
            );
25         } catch (JMException e) {
26             throw new IllegalStateException("Unable to register
                lab4 MBeans", e);
27         }
28     }
29 }

```



```

28     }
29
30     public static synchronized void unregisterMBeans() {
31         MBeanServer server = ManagementFactory.
32             getPlatformMBeanServer();
33         unregister(server, POINT_STATISTICS_OBJECT_NAME);
34         unregister(server, AREA_OBJECT_NAME);
35     }
36
37     public static void registerPoint(boolean hit, double radius) {
38         POINT_STATISTICS.registerPoint(hit, radius);
39     }
40
41     public static void updateRadius(double radius) {
42         STATE.updateRadius(radius);
43     }
44
45     private static void register(MBeanServer server, ObjectName
46         objectName, Object mbean) throws JMException {
47         if (server.isRegistered(objectName)) {
48             server.unregisterMBean(objectName);
49         }
50         server.registerMBean(mbean, objectName);
51     }
52
53     private static void unregister(MBeanServer server, String
54         objectName) {
55         try {
56             ObjectName name = new ObjectName(objectName);
57             if (server.isRegistered(name)) {
58                 server.unregisterMBean(name);
59             }
60         } catch (JMException ignored) {
61             // Application shutdown must not be blocked by JMX
62             cleanup.
63         }
64     }
65 }

```

Listing 8: MBeanRegistrationListener.java

```

1 package com.example.lab3.mbean;
2
3 import jakarta.servlet.ServletContextEvent;
4 import jakarta.servlet.ServletContextListener;
5 import jakarta.servlet.annotation.WebListener;
6
7 @WebListener
8 public class MBeanRegistrationListener implements
9     ServletContextListener {
10
11     @Override

```

```

11     public void contextInitialized(ServletContextEvent sce) {
12         MonitoringRegistry.registerMBeans();
13         sce.getServletContext().log("Lab4 monitoring MBeans
            registered");
14     }
15
16     @Override
17     public void contextDestroyed(ServletContextEvent sce) {
18         MonitoringRegistry.unregisterMBeans();
19         sce.getServletContext().log("Lab4 monitoring MBeans
            unregistered");
20     }
21 }

```

5.5. Интеграция с основным приложением

После вычисления результата проверки точки вызывается `MonitoringRegistry`. При изменении радиуса вызывается `MonitoringRegistry.updateRadius`.

Listing 9: AreaCheckBean.java

```

1 package com.example.lab3.beans;
2
3 import com.example.lab3.model.CheckResult;
4 import com.example.lab3.mbean.MonitoringRegistry;
5 import com.example.lab3.service.AreaHitService;
6 import com.example.lab3.util.Messages;
7 import com.google.gson.Gson;
8 import com.google.gson.GsonBuilder;
9 import com.google.gson.JsonPrimitive;
10 import com.google.gson.JsonSerializer;
11 import jakarta.annotation.PostConstruct;
12 import jakarta.enterprise.context.SessionScoped;
13 import jakarta.faces.context.FacesContext;
14 import jakarta.inject.Named;
15 import jakarta.persistence.EntityManager;
16 import jakarta.persistence.PersistenceContext;
17 import jakarta.transaction.Transactional;
18
19 import java.io.Serializable;
20 import java.time.LocalDateTime;
21 import java.util.Collections;
22 import java.util.List;
23 import java.util.Map;
24
25 @Named("areaCheckBean")
26 @SessionScoped
27 public class AreaCheckBean implements Serializable {
28     private static final String REQUEST_PARAM_X_KEY = Messages.get
        ("request.param.x");

```

```

29     private static final String REQUEST_PARAM_Y_KEY = Messages.get
        ("request.param.y");
30     private static final String RESULTS_QUERY = Messages.get("query
        .results.desc");
31
32     @PersistenceContext(unitName = "default")
33     private EntityManager entityManager;
34
35     private Double x = 0.0;
36     private Double y = 0.0;
37     private Double r = 1.5;
38
39     private List<CheckResult> results;
40
41     private final transient Gson gson = new GsonBuilder()
42         .registerTypeAdapter(LocalDate.class, (
43             JsonSerializer<LocalDateTime>) (src, typeOfSrc,
44             context) ->
45                 new JsonPrimitive(src.toString()))
46         .create();
47
48     @PostConstruct
49     public void init() {
50         loadResults();
51     }
52
53     @Transactional
54     public void check() {
55         processCheck(this.x, this.y);
56     }
57
58     @Transactional
59     public void checkFromCanvas() {
60         Map<String, String> params = FacesContext.
61             getCurrentInstance().getExternalContext().
62             getRequestParameterMap();
63         try {
64             double canvasX = Double.parseDouble(params.get(
65                 REQUEST_PARAM_X_KEY));
66             double canvasY = Double.parseDouble(params.get(
67                 REQUEST_PARAM_Y_KEY));
68             processCheck(canvasX, canvasY);
69         } catch (NumberFormatException | NullPointerException e) {
70             System.err.println(Messages.get("error.canvas.parse.
71                 prefix") + e.getMessage());
72         }
73     }
74
75     private void processCheck(double x, double y) {
76         CheckResult result = new CheckResult();
77         result.setX(x);
78         result.setY(y);

```

```

72     result.setR(this.r);
73     result.setHit(AreaHitService.isHit(x, y, this.r));
74     result.setRequestTime(LocalDateTime.now());
75
76     entityManager.persist(result);
77     MonitoringRegistry.registerPoint(result.isHit(), result.
78         getR());
79     if (results != null) {
80         results.add(0, result);
81     }
82
83     private void loadResults() {
84         try {
85             this.results = entityManager.createQuery(RESULTS_QUERY,
86                 CheckResult.class)
87                                     .getResultList();
88         } catch (Exception e) {
89             this.results = Collections.emptyList();
90             System.err.println(Messages.get("error.results.load.
91                 prefix") + e.getMessage());
92         }
93     }
94
95     public String getResultsAsJson() {
96         if (results == null) {
97             return Messages.get("json.empty.array");
98         }
99         return gson.toJson(results);
100     }
101
102     public Double getX() { return x; }
103     public void setX(Double x) { this.x = x; }
104     public Double getY() { return y; }
105     public void setY(Double y) { this.y = y; }
106     public Double getR() { return r; }
107     public void setR(Double r) {
108         this.r = r;
109         if (r != null) {
110             MonitoringRegistry.updateRadius(r);
111         }
112     }
113     public List<CheckResult> getResults() { return results; }
114     public void setResults(List<CheckResult> results) { this.
115         results = results; }
116 }

```

6. Расчёт площади области

В лабораторной работе №3 область состоит из трёх частей:

- прямоугольника в четвёртой четверти с шириной r и высотой $r/2$;
- четверти круга во второй четверти с радиусом $r/2$;
- треугольника в третьей четверти с катетами r и $r/2$.

Площади частей равны:

$$S_1 = \frac{r^2}{2}, \quad S_2 = \frac{\pi r^2}{16}, \quad S_3 = \frac{r^2}{4}.$$

Итоговая площадь:

$$S = S_1 + S_2 + S_3 = r^2 \left(0.75 + \frac{\pi}{16} \right).$$

Например, для $r = 1.5$:

$$S = 1.5^2 \left(0.75 + \frac{\pi}{16} \right) \approx 2.129286.$$

Для $r = 2$:

$$S = 2^2 \left(0.75 + \frac{\pi}{16} \right) \approx 3.785398.$$

7. Мониторинг приложения в JConsole

7.1. Регистрация и атрибуты MBean

MBean регистрируются со следующими именами:

- `com.example.lab4:type=PointStatistics`;
- `com.example.lab4:type=Area`.

MBean `PointStatistics` предоставляет атрибуты `TotalPoints`, `HitPoints`, `MissPoints` и операцию `reset()`. MBean `Area` предоставляет атрибуты `Radius` и `Area`.

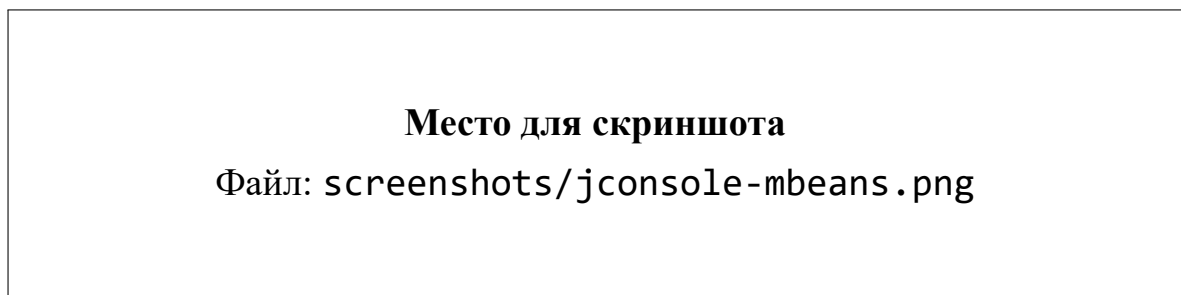


Рис. 1: MBean и их атрибуты в JConsole

7.2. Проверка JMX-уведомления

Для проверки уведомления были выполнены следующие действия:

1. в JConsole открыт MBean `com.example.lab4:type=PointStatistics`;
2. до добавления десятой точки проверено отсутствие уведомления;
3. в веб-приложении добавлены точки до общего количества 10;
4. в JConsole проверено появление уведомления типа `com.example.lab4.point`.

В контрольном запуске после десяти точек получены значения `TotalPoints=10`, `HitPoints=7`, `MissPoints=3`. Уведомление отправлено один раз — в момент перехода счётчика с 9 на 10.

Место для скриншота

Файл: `screenshots/jconsole-notification.png`

Рис. 2: Уведомление после добавления десятой точки

7.3. Время работы виртуальной машины

В стандартном MBean `java.lang:type=Runtime` в JConsole был снят атрибут `Uptime`. Значение измеряется в миллисекундах и показывает время, прошедшее с момента запуска JVM.

Место для скриншота

Файл: `screenshots/jconsole-uptime.png`

Рис. 3: Атрибут `Uptime` стандартного `RuntimeMXBean`

8. Мониторинг и профилирование в VisualVM

В VisualVM был выбран процесс Java-сервера с развёрнутым приложением. После добавления точек изменялись атрибуты `TotalPoints`, `MissPoints` и `Area`. При выборе другого радиуса изменялся атрибут `Area`.

Место для скриншота

Файл: `screenshots/visualvm-mbeans-graph.png`

Рис. 4: График изменения показателей MBean во времени в VisualVM

Для анализа памяти использовался раздел `Sampler/Profiler`. В исходной реализации список результатов хранится в объекте с областью видимости `SessionScope` и при каждой проверке пополняется новым объектом `CheckResult`. Поэтому при большом количестве запросов размер списка и количество удерживаемых объектов растут.

Место для скриншота

Файл: `screenshots/visualvm-memory-before.png`

Рис. 5: Результат анализа памяти до исправления

В качестве проблемы производительности рассматривается не само сохранение результата в базе данных, а неограниченное накопление всех результатов в сессионном списке. Для устранения проблемы следует ограничить размер отображаемой истории либо заменить полную загрузку на постраничную выборку из базы данных. После внесения исправления необходимо повторить профилирование и приложить результат сравнения.

Место для скриншота

Файл: `screenshots/visualvm-memory-after.png`

Рис. 6: Результат анализа памяти после исправления

9. Локализация и устранение проблемы

9.1. Алгоритм локализации

Для воспроизведения поиска проблемы выполняются следующие действия:

1. запустить приложение и подключиться к его JVM в VisualVM;
2. открыть Sampler или Profiler и выполнить снимок распределения памяти;
3. зафиксировать классы, занимающие наибольший объём памяти;
4. выполнить большое количество проверок точек;
5. повторить снимок и сравнить количество объектов и занимаемый ими объём;
6. проверить ссылки на удерживаемые объекты и найти коллекцию, которая продолжает расти;
7. ограничить историю результатов или реализовать пагинацию;
8. повторить измерение и убедиться, что рост памяти устранён или ограничен.

Наиболее вероятным источником роста памяти является `java.util.ArrayList`, используемый полем `results` класса `AreaCheckBean`, и объекты `com.example.lab3` добавляемые в этот список.

Отдельно предусмотрена очистка `MBean` при остановке приложения. В `Monitorin` перед регистрацией проверяется наличие объекта с тем же именем, а `MBeanRegistrat` снимает `MBean` с регистрации. Это предотвращает конфликт имён при повторном развёртывании приложения и удержание старого экземпляра `MBean`.

Место для скриншота

Файл: `screenshots/visualvm-profiling.png`

Рис. 7: Локализация проблемы в VisualVM и проверка результата исправления

10. Проверка сборки и работоспособности

Проект собирается под Java 17. Для проверки необходимо использовать JDK 17, Maven и сервер приложений Jakarta EE, например WildFly, настроенный для работы с данным приложением и источником данных.

Минимальная последовательность проверки:

1. выполнить сборку WAR-файла командой `mvn clean package`;
2. развернуть `target/lab4.war` на сервере приложений;
3. открыть веб-приложение и добавить несколько точек;
4. проверить регистрацию двух MBean в JConsole;
5. добавить десятую точку и проверить уведомление;
6. подключиться к тому же процессу в VisualVM и выполнить мониторинг и профилирование.

Проверка исходной логики попадания выполняется тестами `AreaHitServiceTest`. Для окончательной проверки лабораторной необходимо также сохранить результаты JConsole и VisualVM в каталоге `report/screenshots` и заменить заполняющие блоки настоящими изображениями.

11. Вывод

В лабораторной работе реализованы пользовательские MBean для мониторинга Java-веб-приложения. MBean `PointStatistics` считает общее количество точек и количество промахов, а также отправляет уведомление после каждой десятой точки. MBean `Area` предоставляет текущий радиус и площадь фигуры.

С помощью JConsole проверяются атрибуты разработанных MBean, JMX-уведомление и время работы JVM. С помощью VisualVM выполняются наблюдение за изменением показателей, анализ распределения памяти и локализация проблемы неограниченного накопления объектов истории проверок. После ограничения истории или реализации пагинации необходимо повторно выполнить профилирование и подтвердить устранение проблемы скриншотами.